electric-eye: Real-Time Image Enhancement and Low-Latency Video Transport for Remotely Operated Underwater Vehicles

H. Oğuz Bolat[a,*], Hakan Ersoy[b]

[a]Department of Geomatics Engineering, Yıldız Technical University, Davutpaşa Campus, 34220 Esenler, İstanbul, Türkiye
[b]Department of Electronics and Communication Engineering, Yıldız Technical University, Davutpaşa Campus, 34220 Esenler, İstanbul, Türkiye

ARTICLE INFO

Keywords:
Remotely operated vehicle
Underwater imaging
Image enhancement
Low-latency video
RTP
Embedded Linux

ABSTRACT

The pilot of a remotely operated vehicle steers by a video picture that turbid water degrades and that a tether carries with delay, and image processing inserted between the camera and the pilot must add as little delay as it can. We describe electric-eye, two C99 programs for GNU/Linux that sit between an ROV's camera and its pilot. `eeye` applies a chain of image operators (point operations fused into look-up tables, blur, Sobel, Laplacian-of-Gaussian and Canny edges) and an optional continuous visibility correction (cast removal, contrast stretch and contrast-limited adaptive histogram equalization, smoothed over time), and publishes frames on a shared-memory frame bus that never waits for its readers. `hellion` carries the picture across the tether as H.264 over RTP, relays the vehicle's packets to QGroundControl unchanged, records them, measures the link, serves a browser control page, and finds its counterpart on the network by itself. Either program can run on either side of the tether. Two changes on the receiving side, an end-of-frame delimiter inserted by the relay and slice-threaded decoder output, reduced the transport latency from bus to bus over loopback from 70.4 to 3.8 ms at 640×480 and from 74.8 to 8.3 ms at 1280×720. The visibility correction costs 0.55 ms per 1280×720 frame on four desktop threads. Optimizations that should not change the output were checked byte for byte against the previous code on x86-64 and, under emulation, on ARMv7 and AArch64. The software has not yet been run on a vehicle; code and release binaries are open.

1. Material and methods

1.1. System overview

The pipeline processes the video of a remotely operated vehicle (ROV) between its camera and the pilot. It was built for a BlueROV2 (Blue Robotics, 2026), whose onboard Raspberry Pi runs BlueOS and the ArduSub autopilot (ArduPilot Dev Team, 2026) and is joined to a topside computer by an Ethernet tether, and it runs on any GNU/Linux machine. It consists of two programs (Figure 1). `eeye`, the vision layer, takes frames from a camera or from the tether, applies a configurable chain of image operators and an optional continuous visibility correction, and publishes the result (*pilot*) and the untouched camera frame (*raw*) on a shared-memory *frame bus* for any other program on the machine; it has no network code. `hellion` does everything that crosses the tether: it encodes the video as H.264 (ITU-T, 2003) over RTP (Schulzrinne et al., 2003; Wang et al., 2011), receives it topside, relays it to QGroundControl (Dronecode Foundation, 2026), records it, serves a browser control page, and carries `eeye`'s settings from the page to wherever `eeye` runs. The two programs communicate only through the frame bus and `eeye`'s configuration file, so either can be restarted without the other.

*Corresponding author
oguz.bolat@std.yildiz.edu.tr (H.O. Bolat)
ORCID(s): 0009-0000-8522-7746 (H.O. Bolat)

eeye runs on one side of the tether, chosen at installation (the *placement*; Figure 1). In placement B it runs on the vehicle next to the camera, the vehicle sends the processed picture, and the topside relays the received packets to QGroundControl unchanged, so the picture is encoded once. In placement A the vehicle sends its camera, eeye runs topside on the decoded stream, and the processed picture is encoded a second time for QGroundControl; this moves the image processing off the vehicle's processor. Autonomy code is expected to run next to eeye and read *raw* from the bus, so that its input is the camera's frame and carries the camera's timestamps.

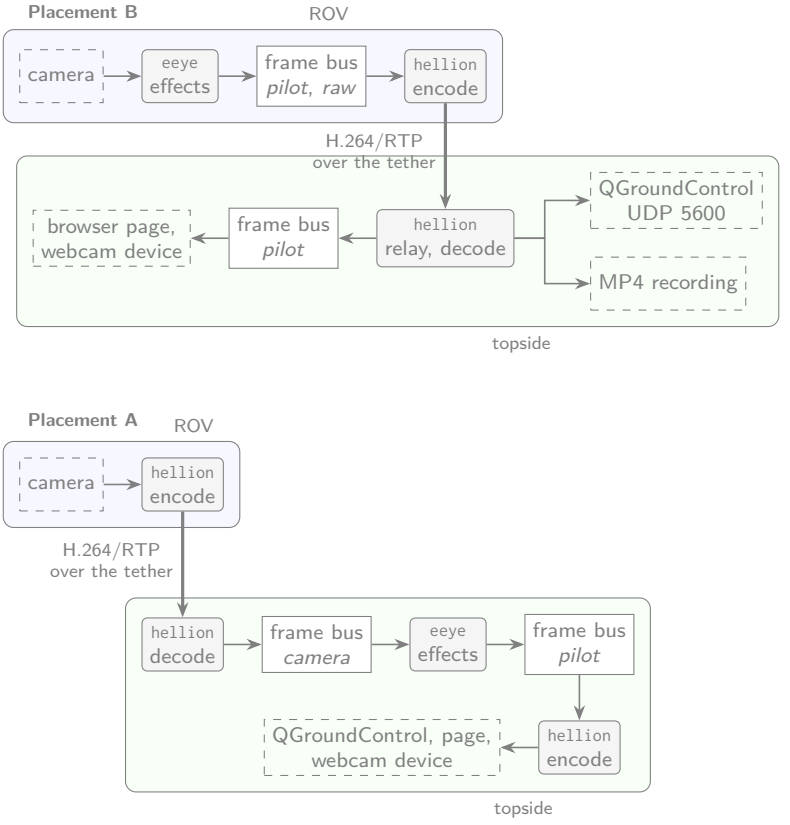


Figure 1: The two programs and the two placements. In placement B, used on the vehicle, eeye processes the camera on the ROV, and QGroundControl receives the ROV's packets unchanged. In placement A the ROV sends its camera and the processing runs topside, at the cost of a second encode. Programs on one machine exchange frames only through the frame bus.

1.2. Image acquisition

eeye captures from a USB (UVC) camera through V4L2 (The Linux kernel developers, 2026) with memory-mapped streaming, or from a Raspberry Pi camera module through a `rpicam-vid` (Raspberry Pi Ltd, 2026) child process; a third source reads a frame bus stream (placement A). On V4L2 it takes MJPEG when the camera offers it at the requested size and packed 4:2:2 (YUYV) otherwise, because at USB 2.0 bandwidth most cameras offer 720p only as MJPEG. When the requested size is not offered, the closest mode is chosen by aspect ratio first and pixel count second, once

at startup, since all frame buffers are allocated at that size. MJPEG is decoded with libjpeg-turbo (The libjpeg-turbo Project, 2026) directly into 4:2:2 planes; an optional downscale by 2, 4 or 8 is applied inside the decoder (by dropping DCT coefficients) or, for YUYV, while the packed samples are deinterleaved.

Every frame is held as planar I422 at full range (0–255 for luma and chroma, the range the operators assume; ITU-R, 2011). It carries a frame number and its capture time on `CLOCK_MONOTONIC`: the driver's buffer timestamp when the driver stamps that clock, otherwise the time of receipt. Each capture takes, after the first ready buffer, every further buffer the driver has filled, returns the older ones unread and decodes only the newest; when all buffers fill, the V4L2 driver keeps the oldest frames and drops new ones, and without this a single slow frame left the picture permanently several frames behind.

Camera controls (exposure, gain, white balance, brightness, contrast, saturation, sharpness) are expressed in device-independent units, percentages of the device's reported range or physical units (exposure in microseconds, white balance in kelvin), and are mapped onto each camera's own range. A field left out leaves that control untouched. One control is set by default: the V4L2 control that lets automatic exposure lengthen the shutter beyond the frame period is turned off, because with it on the laptop camera used for testing (Section 1.9) fell from 30 to 17 frames per second in a dim room, and every later stage inherits the rate. In placement A the controls travel from the topside to the vehicle's `hellion`, which holds the camera, applies them, and keeps them for its next start.

1.3. Processing threads and buffers

Inside `eeye` a producer thread captures, an effects thread runs the operators, and a consumer thread publishes (Figure 2). The threads pass frames through three ring buffers of 32 slots, and frames come from a pool of eight that is allocated once and never grows, so that the pool size bounds the lag between capture and output (eight frames, 267 ms at 30 frames per second). Each frame has three planar buffers: the untouched capture (*raw*), the working image, and a spare that lets two consecutive neighbourhood operators alternate without copying.

The configuration is a JSON file that a watcher thread reloads within milliseconds of any change (inotify). A parsed configuration is published by an atomic pointer swap, and the effects thread takes one snapshot per frame, so that no frame is processed with two configurations. After every load `eeye` writes whether it accepted the file; `hellion`, which writes the file for the browser page, waits for that verdict and puts the previous file back when the new one is refused. Once a second the consumer assembles a health record (capture state, frame count, recording state and the image statistics of Section 1.5); the stats thread writes it, so that file input/output on the vehicle's SD card cannot stall the thread that publishes the pilot's picture.

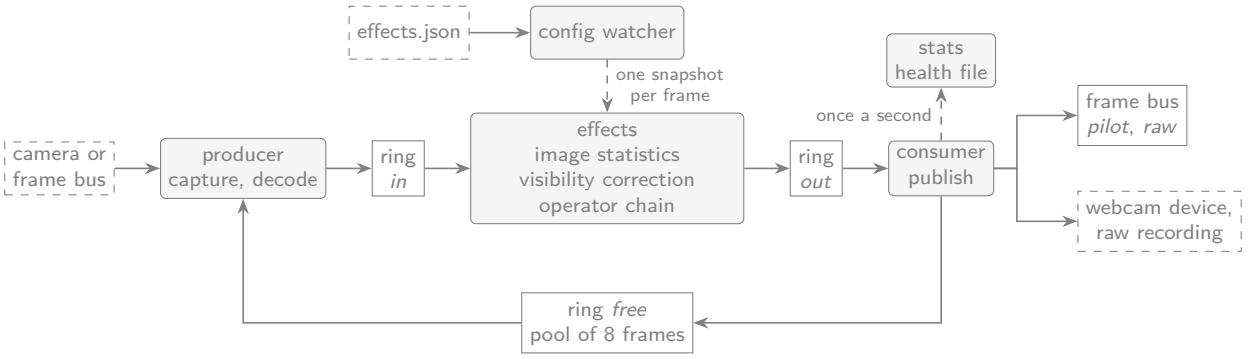


Figure 2: Threads and buffers of eeye. Frames circulate from the free ring through capture, processing and publication and back; the pool of eight frames bounds the output lag. Configuration changes reach the effects thread as whole snapshots. The stats thread writes the health file, so that file input/output never runs on the consumer.

1.4. Image operators

The operator chain holds up to eight stages applied in order; Table 1 lists the operators. Point operators (grayscale, invert, threshold, tint, light) are composed into one 256-entry lookup table per plane for each run of consecutive point stages, so that a run costs one pass over the frame whatever its length. The neighbourhood operators read 3×3 or 5-sample neighbourhoods and replicate the border samples, because zero padding would produce a false response along the frame edge.

The Gaussian blur repeats a separable binomial kernel,

$$g(x) = \left\lfloor \frac{p_{x-2} + 4p_{x-1} + 6p_x + 4p_{x+1} + p_{x+2} + 8}{16} \right\rfloor, \tag{1}$$

horizontally and then vertically, s times for strength s; repeated passes widen the effective Gaussian as $\sqrt{s}$. Each thread computes its own band of output rows in a single sweep, keeping the five horizontally filtered rows it needs in a rolling window, and successive passes alternate between the destination and a scratch plane, which needs one synchronization of the thread team per pass where filtering in place needs two. From $s = 4$ the passes run at half resolution: the plane is reduced by 2×2 averaging, blurred, and expanded by bilinear interpolation with weights $\frac{3}{4}$ and $\frac{1}{4}$.

Sobel (Duda and Hart, 1973) computes the gradients G_x and G_y of the 3×3 Sobel kernels and outputs $\min(|G_x| + |G_y|, 255)$, set to zero below a threshold. The Laplacian-of-Gaussian operator (Marr and Hildreth, 1980) blurs with Eq. (1), takes the 4-neighbour Laplacian in 16-bit signed arithmetic and marks a pixel where the response changes sign towards its right or lower neighbour by at least a threshold; comparing in two directions only keeps contours one pixel wide, and the slope test rejects noise that merely grazes zero. Canny's detector (Canny, 1986) is implemented in four stages: at least one blur pass, which keeps the cost of hysteresis bounded on noisy input; the Sobel gradient, with its direction quantized to four sectors by integer comparisons ($|G_y| < 0.4|G_x|$ horizontal, $|G_y| > 2.4|G_x|$ vertical,

otherwise diagonal by the signs of G_x and G_y); non-maximum suppression, keeping a pixel when it is not below one neighbour across the edge and strictly above the other, which thins a symmetric ridge to one pixel; and hysteresis. Suppression labels each pixel directly in the output, as a seed when its magnitude reaches the high threshold and as a candidate when it reaches the low one; each row's seeds are collected as it is labelled; a serial flood fill from the seeds promotes connected candidates, and a final parallel pass clears the candidates it did not reach. The flood fill gives the exact connected set, which a parallel sweep would only approximate.

The edge operators output luma only and set the chroma planes to the neutral value 128. A pass that computes chroma no later stage will read (a blur, tint or contrast stretch before an edge operator, for example) is skipped; the stages after the last one that writes chroma are traced back once per configuration. When a chain's output chroma is known to be neutral, the frame is marked and the frame bus copies only luma (Section 1.6).

The neighbourhood operators have NEON implementations, written with intrinsics that exist on both ARMv7 and AArch64, and portable scalar versions that the compiler vectorizes on x86. Loops are parallelized with OpenMP (Dagum and Menon, 1998); every parallel loop is free of data races, so that it is also correct without OpenMP. Each vector or parallel path was required to produce output byte-identical to the scalar single-threaded path (Section 1.9).

Table 1
Image operators of the chain. All parameters are integers from 0 to 255.

Operator	Parameters (default)	Operation	Planes
grayscale	–	chroma set to 128	UV
invert	–	$Y \leftarrow 255 - Y$	Y
threshold	level (0)	$Y \leftarrow 255$ if $Y \geq$ level, else 0	Y
tint	u, v, strength (0)	chroma moved towards (u, v) by strength/255	UV
light	level (128)	$Y \leftarrow Y +$ level $- 128$; chroma scaled about 128 by level/128	YUV
contrast	–	per-frame min/max stretch of Y to 0–255	Y
blur	strength s (0, one pass)	Eq. (1), s passes; half resolution from $s = 4$	YUV
sobel	threshold (0)	$\min(\lvert G_x\rvert + \lvert G_y\rvert, 255)$, zero below threshold	Y; UV $\leftarrow$ 128
log	strength (0, one pass), threshold (0)	blur, 4-neighbour Laplacian, zero-crossings with slope test	Y; UV $\leftarrow$ 128
canny	strength (0, one pass), low (40), high (90)	blur, gradient, suppression, hysteresis	Y; UV $\leftarrow$ 128

Note. Consecutive point operators (the first five rows) are fused into one lookup table per plane. The image statistics and the visibility correction of Section 1.5 run before the chain and are not chain stages.

1.5. Continuous visibility correction

Water absorbs red light first, scatters light into the line of sight, and attenuates contrast with distance (Jaffe, 1990; Schettini and Corchs, 2010): underwater pictures are blue-green, compressed in brightness and hazy, and they change as the vehicle descends. Because the pilot cannot tune effects while flying, `eeye` provides a correction, off by default and switched on from a single control on the page, that adapts continuously and has no parameters to set. It runs before the operator chain, on the pilot's picture only; *raw* is never altered, so that autonomy code and the pilot's own

comparison see the camera's frame.

The correction is steered by statistics of the raw frame, taken on every second pixel of every second row: the luma histogram, and the mean chroma offsets $\bar{u}$, $\bar{v}$ from neutral over samples with $24 \le Y \le 235$, which excludes pixels too dark to carry colour and pixels that have lost it to clipping. From these it derives three corrections, each a target that the applied correction follows with the first-order filter

$$c_k = c_{k-1} + \alpha_k \, (c_k^* - c_{k-1}), \qquad \alpha_k = 1 - e^{-\Delta t_k / \tau}, \tag{2}$$

where Δt_k is the difference between the capture times of consecutive frames, so that the response does not depend on the frame rate. A gap of more than 1 s, or switching the correction on, sets $c_k = c_k^*$, without fading in from no correction.

Colour cast. Following the grey-world assumption (Buchsbaum, 1980), the target chroma shift removes the measured cast down to a deadband d,

$$(\Delta u^*, \Delta v^*) = -\frac{\max(m - d, 0)}{m} \, (\bar{u}, \bar{v}), \qquad m = \sqrt{\bar{u}^2 + \bar{v}^2}, \tag{3}$$

with each component limited to ± 48. The deadband leaves the small cast that a scene's own colours produce; without it the synthetic scene of Figure 3 (red, yellow and orange objects, cast 9.9) had its grey background shifted by 11.7 chroma units towards blue. Since a cast is a gain on the light, the shift applied at a chroma sample is scaled by $\min(Y / \bar{Y}, 1.5)$, where Y is the luma under the sample and $\bar{Y}$ the mean luma of the measured samples, and chroma is then scaled about 128 by a saturation gain of 1.2.

Brightness range. The 0.5th and 99.5th luma percentiles, p_{lo} and p_{hi}, are stretched to 0 and 255; percentiles rather than extrema, so that a few bright particles of backscatter cannot defeat the stretch. When $p_{\mathrm{hi}} - p_{\mathrm{lo}}$ is under 96 levels the interval is widened to 96 about its centre, which limits the gain to 2.7 and keeps the sensor noise of a flat, dark frame from being amplified into texture. A gamma $\gamma = \ln 0.45 / \ln \tilde{m}$, limited to 0.55–1.6, where $\tilde{m}$ is the median luma after the stretch, places the median at 45% of full scale.

Haze. Contrast-limited adaptive histogram equalization (CLAHE; Pizer et al., 1987; Zuiderveld, 1994), applied to luma as in earlier underwater enhancement (Hitam et al., 2013), restores local contrast. The frame is divided into tiles of about 160×120 pixels (at most 8×6); each tile's histogram is clipped at twice the height of a flat histogram, the excess is spread evenly over all levels, and the cumulative histogram gives the tile's mapping, mixed with the identity at a weight of 0.7. The tile mappings are smoothed over time by Eq. (2), composed with the global stretch, and interpolated bilinearly between the four nearest tile centres for each pixel. The tile histograms are taken from the source luma and pushed through the global mapping, so that the globally corrected image need not be written and read again. Table 2

lists the constants.

The correction was evaluated on synthetic frames formed by the classical underwater image model (Jaffe, 1990), a simplification of the revised model of Akkaynak and Treibitz (2018),

$$I_c = J_c\, t_c + B_c\,(1 - t_c), \qquad c \in \{R, G, B\}, \tag{4}$$

with transmissions $t = (0.25, 0.70, 0.80)$ and veiling light $B = (15, 110, 125)$, applied to a scene of coloured patches, gratings of three spatial frequencies, a gradient and noise of ± 2 levels (Figure 3). After 30 frames the cast magnitude m fell from 39.1 to 9.1, the 1st–99th percentile luma range rose from 129 to 249 levels, and the mean absolute difference between neighbouring luma samples, a measure of visible edge contrast, rose from 7.86 to 19.17. The red patch, grey-green in the degraded frame ($V - 128 = 0.0$), read red again (23.1). On a uniform frame of luma 30 ± 3, the standard deviation rose from 2.00 to 7.80 (3.9×). After a step change in brightness the first frame differed from the settled result by 105.9 levels in mean luma and the result settled within 1.5 s; a still scene produced consecutive frames that differ by at most one level in fewer than 0.1% of pixels.

Table 2
Constants of the visibility correction. They were set on the synthetic frames of Eq. (4) and a laptop camera, and are to be revised on footage from the vehicle.

Constant	Value
Smoothing time constant τ, Eq. (2)	0.4 s
Gap that restarts the smoothing	1 s
Luma range of samples used for the cast	24–235
Cast deadband d, Eq. (3)	8
Largest chroma shift	±48
Saturation gain	1.2
Stretch percentiles	0.5th, 99.5th
Smallest stretched interval (largest gain)	96 levels (2.7)
Target median	0.45
Gamma limits	0.55–1.6
CLAHE tile size (largest grid)	≈160×120 px (8×6)
CLAHE clip limit	2.0
Weight of the local mapping	0.7

1.6. Frame bus

Programs on one machine exchange video through the frame bus: one shared-memory file per stream (in `/dev/shm`), with a 4096-byte header and four slots (Table 3). The writer never waits for a reader. It writes frame n into slot $(n - 1) \bmod 4$, protected by a sequence lock (Boehm, 2012): the slot's counter is odd while it is written and $2n$ when frame n is complete. A reader takes the newest frame number, copies the slot, and keeps the copy only if the counter read $2n$ before and after; otherwise the writer overtook it and it starts over. A slow reader therefore skips frames and

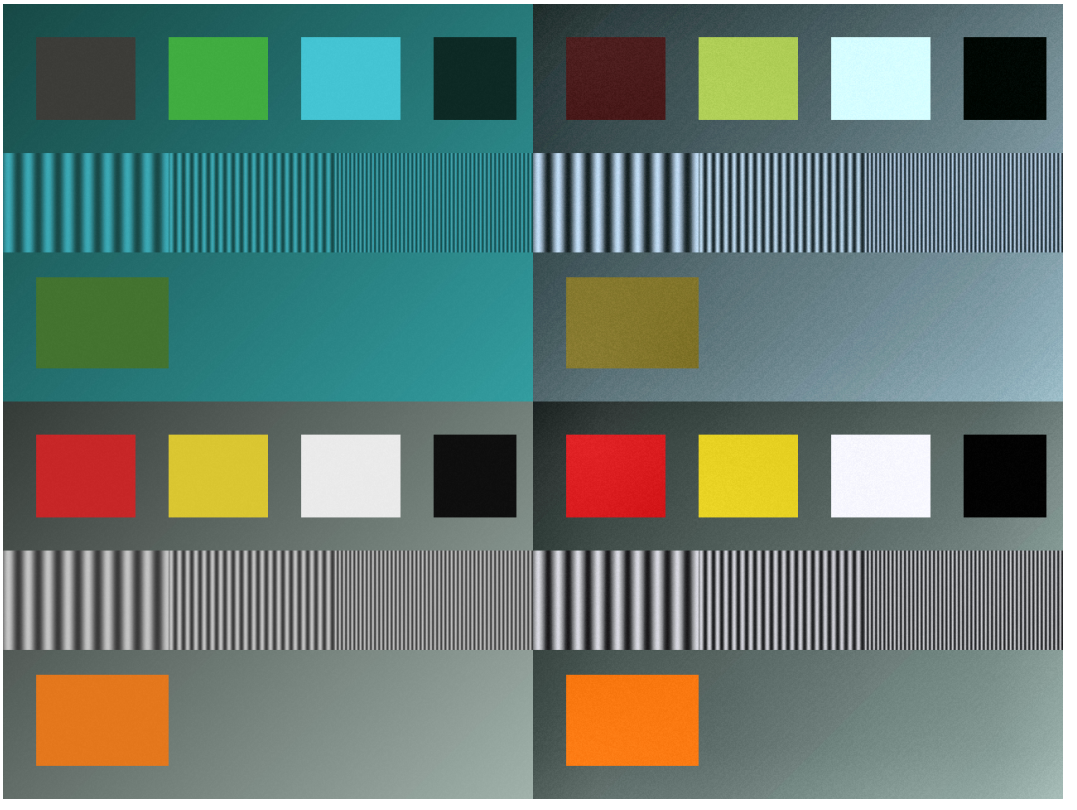


Figure 3: The visibility correction on synthetic frames. Top: the scene degraded by Eq. (4) (left) and corrected (right). Bottom: the undegraded scene (left) and corrected (right), where the correction should change little. The red, yellow and orange patches lose most of their red to the model's transmission of 0.25 and recover only partly.

cannot delay the pilot's picture. Readers sleep on a futex (Franke et al., 2002) that the writer wakes after each frame. A restarted writer builds a new file and renames it into place; readers notice the new file and map it again. A heartbeat updated every 500 ms tells a stopped writer from a live one with no new frames, after 3 s.

A frame marked as having neutral chroma (Section 1.4) is published by copying luma only; each slot's chroma planes are filled with 128 the first time the slot receives such a frame and left in place afterwards, so that readers receive the same bytes as from a full copy. The writer keeps the layout it created in its own memory rather than reading it back from the shared header, so that a header damaged by another process with access to the file cannot direct the writer's copies outside the mapping. The layout is documented so that a reader can be written in any language; Python needs only `mmap` and `struct`.

1.7. Video transport across the tether

`hellion` runs *pipelines*, each with one source (a camera, a frame bus stream, or RTP from the tether) and up to eight sinks (a frame bus stream, a v4l2loopback webcam device (zmölnig, 2026), or RTP); each sink has its own thread behind a one-frame mailbox in which the newest frame replaces an unsent one, so that a slow sink loses its own frames

Table 3
Frame bus layout. Offsets in bytes; all fields little-endian.

Offset	Field	Content
Header (4096 bytes)		
0	magic	`EEYEBUS1`
16	generation	differs for every writer run
24, 28	width, height	image size in pixels
32, 36	format, range	I422 planar (`422P`); full range
40–75	plane offsets, strides, sizes	Y, U, V within a slot's image data
76, 80	slot count, slot header size	4; 64 bytes
128	futex word	changes on every frame and on close
136	write sequence	number of frames published
152	heartbeat	monotonic time, renewed every 500 ms
Slot header (64 bytes), followed by the Y, U and V planes		
0	sequence	$2n$ when frame n is complete; odd while written
8	frame number	counts one per captured frame
16, 24	capture time	monotonic and wall-clock nanoseconds
32	publish time	monotonic nanoseconds

without delaying the others (Figure 4). Every sink but the last receives a copy of the frame; the last receives the frame's own buffer in exchange for its mailbox buffer, so that a pipeline with one sink, as on the vehicle, copies no frames.

H.264 encoding and decoding run in FFmpeg child processes (FFmpeg developers, 2026) (Table 4). The encoder receives raw frames on a pipe and sends RTP to a local port. Before the pipe, `hellion` converts the bus's full-range 4:2:2 frame to the limited-range 4:2:0 frame that H.264 receivers expect,

$$Y' = 16 + \frac{219}{255}\,Y, \qquad C' = 128 + \frac{224}{255}\,(C - 128), \tag{5}$$

in 16-bit fixed point after averaging chroma rows in pairs, and the decoder applies the inverse; uniform frames of luma 40 and 200 sent across the tether return as 40 and 200. Converting in `hellion` rather than in an FFmpeg filter reduced the encoder's CPU time by 29% at 1280×720 and 30 frames per second and the bytes on the pipe by 25%. Frames are time-stamped at the configured nominal rate; stamping them with their arrival time made FFmpeg drop one frame in six at its default time base and, at a 1 ms time base, made the x264 rate control spend 161 kbit/s of a 4 000 kbit/s budget.

All RTP traffic passes through a relay in `hellion`. On the vehicle the encoder's local port is relayed to the topside; topside, the relay owns the tether port and copies every packet to the local decoder, to the recorder and to QGroundControl. The relay sees every RTP header, so that each link's bit rate and loss are measured from the sequence numbers (modulo 2^{16}); a late packet is not counted as lost, a new synchronization source starts a new count, and more than 1 s of silence also restarts the count, since a sender that kept running through an outage returns more than 2^{15} numbers ahead. The relay also removes a frame of delay from the decoder: FFmpeg's H.264 parser knows a frame has ended

only when the next one begins, so after each packet that carries the RTP marker bit, which ends a frame (Wang et al., 2011), the relay sends the decoder an access unit delimiter and renumbers the decoder's packets to keep the sequence continuous. QGroundControl and the recorder receive the stream unchanged. The decoder's raw output, a plain copy, runs with slice threading, because the frame-threaded raw video encoder of FFmpeg 5.1 and later releases each frame only when the next one arrives.

The recorder copies the received H.264 into fragmented MP4 without decoding, so that a file cut off by a power loss plays to its last keyframe. When the stream stops, the recording pauses and continues in a new file once packets return. The browser page shows a preview of any frame bus stream, encoded as MJPEG at about 15 frames per second by one encoder per stream that all viewers share, and its control interface carries `eeye`'s settings and camera controls to the side where `eeye` or the camera runs. Once a second the two `hellion` processes exchange a short status description, so that the dashboard on either machine shows what the other end sends and receives. Each `hellion` also announces itself once a second on UDP port 5612, over IPv4 broadcast and IPv6 link-local multicast; a `hellion` configured with automatic addresses pairs with the one of the opposite role that announces the same pair name, and refuses to choose when two answer.

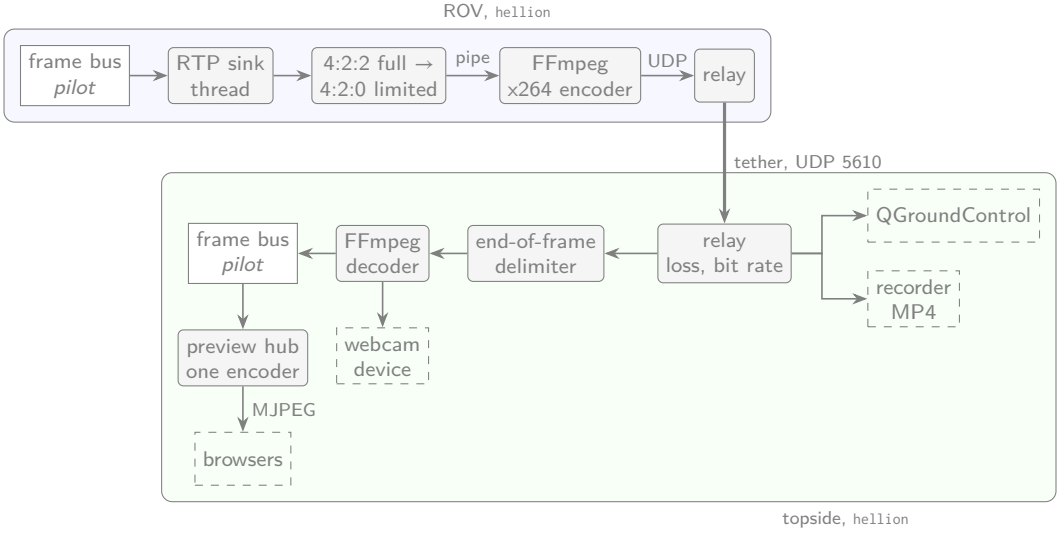


Figure 4: Video transport in placement B. The vehicle's `hellion` converts each frame to the encoder's format and relays the encoder's packets; topside, a relay owns the tether port, measures the link, and copies every packet to QGroundControl and the recorder unchanged and to the decoder followed by an end-of-frame delimiter. One preview encoder per stream serves all browsers.

1.8. Implementation and deployment

Both programs are written in C99 and built by one GNU make file; they need a C99 compiler (GCC 4.7 or later, or Clang), libjpeg-turbo 1.4 or later and, for `hellion`, FFmpeg at run time, and they run on Linux 3.2 or later. Release binaries are built statically for x86-64, ARMv7 with NEON and AArch64 with GCC 15.3 (Bootlin toolchains 2026.08-

Table 4
Transport settings.

Stage	Setting
Encoder input	raw I420, limited range (Eq. 5), nominal frame rate
Encoder	libx264 (VideoLAN, 2026), preset `ultrafast`, tune `zerolatency` (no B-frames, no look-ahead); 4 000 kbit/s, maximum rate 4 000 kbit/s, buffer 2 000 kbit; keyframe every 30 frames; parameter sets before every keyframe
Packetization	RTP, payload of at most 1 200 bytes
Tether	UDP port 5610 (QGroundControl's 5600 is left free)
Decoder input	no input buffering, low-delay flags, no reordering wait; an access unit delimiter after each frame
Decoder output	raw I422, full range, fixed size; slice threading
Recording	stream copy into fragmented MP4, a fragment per keyframe
Preview	MJPEG, 4:2:0, quality 75, ≈15 frames per second, one encoder per stream
Restarts	encoder with back-off from 1 to 30 s; decoder at once, or after 5 s without output while packets arrive

Note. The encoder can be replaced per pipeline, for example by a hardware encoder on the vehicle; the decoder side does not depend on the encoder.

1; Bootlin, 2026) and libjpeg-turbo 3.2.0, so that they need no libraries on the target. Child processes are started with their program resolved before `fork`, with inherited file descriptors closed and signals reset, in their own process group, and are stopped by closing their input before signalling them. An installation script installs both programs as system services on each end of the tether, given the end's role and the placement; the two services run as one service user, so that `hellion` can write `eeye`'s configuration and both can use the frame bus, and neither needs elevated privileges.

1.9. Verification and measurement

The source carries 17 test suites, run on every build: the configuration parser, the ring buffers, the JPEG decoding and framing, resolution negotiation, the operators and the chain, the visibility correction, the camera controls, the raw recorder and the webcam output, child processes, the frame bus under concurrent processes (including a writer overtaking a reader during a copy), the health records, the HTTP server and client, and `hellion`'s pipelines, relay, status file and RTP round trip through FFmpeg. The suites also run, as built for release, on ARMv7 and AArch64 under QEMU user-mode emulation, which executes the NEON paths. The HTTP parsers were fuzzed with libFuzzer, and the suites of the changed components were run under AddressSanitizer and UndefinedBehaviorSanitizer.

Each change that should not alter the output was checked against the previous code by hashing the output of both over a fixed set of inputs: three 1280×720 camera photographs and synthetic frames of edge-case sizes (odd chroma widths, frames one or two pixels wide), every strength and threshold of the operator in steps, 1 to 7 threads, on x86-64 and on both ARM targets under emulation (Table 7). Processing costs were measured as the minimum over repeated runs on an AMD Ryzen 5 5600 (6 cores) with the threads of OpenMP set to sleep while idle, as `eeye` configures them; costs on ARM were estimated as executed instruction counts under QEMU with an instruction-counting plugin, which reflects the work but not the memory behaviour of a Raspberry Pi (Table 6).

Transport latency was measured from frame bus to frame bus through two `hellion` processes, one as the vehicle and one as the topside, connected over loopback on an idle laptop (AMD Ryzen 5 5600H, FFmpeg 9.0.2). A test program published frames whose pixels encode their frame number and read them from the receiving bus; the latency of a frame is the difference between the two times, and an RTP tap between the encoder and the decoder divided it into stages (Table 5). The complete chain was also run between two machines on a local network, the laptop's built-in camera (HP Wide Vision HD, UVC) serving as the vehicle's camera, for placements A and B, for the camera controls in placement A, for the recording and the decoder across disconnections of the network cable, and for automatic pairing. Neither program has yet been run on the vehicle.

Table 5
Transport latency, frame bus to frame bus through two `hellion` processes over loopback; median (10th–90th percentile) of 300 frames.

	640×480 30 frames/s	640×480 60 frames/s	1280×720 30 frames/s
Without the delimiter and slice threading	70.4 (70.2–70.6) ms	37.0 (36.8–37.2) ms	74.8 (74.6–75.1) ms
With both	3.8 (3.7–4.1) ms	3.7 (3.6–3.9) ms	8.3 (8.1–8.6) ms

Note. Without the two measures the decoder held two frame periods: about one in FFmpeg's H.264 parser and one in its raw video output. The encoder, from the frame written to its last RTP packet, took 1.2–1.5 ms at 640×480, and `hellion`'s own relay, pipe and bus about 0.5 ms. Camera exposure, the tether and the vehicle's encoder add to these figures.

Table 6
Processing costs per 1280×720 frame. x86-64: AMD Ryzen 5 5600, minimum over repeated runs, 4 threads unless stated. ARM: executed instructions under emulation, one thread, 20 frames.

Stage	x86-64	ARM (instructions)
Image statistics and visibility correction	0.55 ms (2.09 ms, 1 thread)	–
Blur, strength 1	0.18 ms	AArch64 183 M; ARMv7 263 M
Blur, strength 4	0.59–0.66 ms	AArch64 283 M; ARMv7 391 M
Blur, strength 8	0.84 ms	AArch64 486 M; ARMv7 676 M
Blur, strength 20	1.56 ms	–
Canny, strength 2, thresholds 40/90	3.41–3.82 ms	–
Frame bus publication	34.9 μs; 17.2 μs with neutral chroma	–
`hellion` preview, 1 and 4 viewers	4% and 4% of a core	–
Encoder (FFmpeg, libx264), 30 frames/s	0.21 s of CPU per second of video, laptop	–

Note. Before the measures described in the text, the visibility correction took 1.49 ms, the blur 0.75, 1.38 and 2.27 ms at strengths 1, 4 and 8 (and 900 M and 1 108 M instructions on AArch64 at 4 and 8), Canny 3.80–4.20 ms, the preview 13% of a core with four viewers, and the encoder 0.30 s of CPU per second of video.

Table 7
Equivalence checks: output hashed over all cases for the previous and the revised code; every comparison was identical.

Change	Inputs	Cases	Platforms (threads)
Visibility correction	smooth, flat and noisy synthetic sequences of 60 frames, at sizes from 1920×1080 to 1×1, including padded strides	–	x86-64, AArch64, ARMv7
Canny	3 photographs × 4 strengths × 7 low × 6 high thresholds	504	x86-64 (1, 3, 4), AArch64, ARMv7 (4)
Blur, half-resolution NEON	4 sizes × strengths 0–20 × 3 or 1 planes	168	x86-64 (1, 4), AArch64, ARMv7 (4)
Blur, scratch-plane passes	7 sizes, including 2-pixel-wide frames × strengths 0–20 × 3 or 1 planes	294	x86-64 (1, 3, 4), AArch64, ARMv7 (4)

Code availability

Name of the software: electric-eye (programs `eeye` and `hellion`). Developer: H. Oğuz Bolat, `oguz.bolat@std.yildiz.edu.tr`. Program language: C99. Hardware requirements: an x86-64, ARMv7 with NEON or AArch64 processor; tested on desktop and laptop computers and, for the ARM builds, under emulation. Software required: a C99 compiler, GNU make and libjpeg-turbo to build; FFmpeg at run time for `hellion`. The source code, its documentation and the release binaries are at `https://github.com/hogib/electric-eye`; this description corresponds to release v0.3.1 (commit 96b3eb1).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude Opus 5.5 (Anthropic) to develop parts of the software and to draft and edit the manuscript text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Akkaynak, D., Treibitz, T., 2018. A revised underwater image formation model, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6723–6732.

ArduPilot Dev Team, 2026. ArduSub. `https://ardupilot.org/sub`.

Blue Robotics, 2026. BlueROV2 and BlueOS. `https://bluerobotics.com`.

Boehm, H.J., 2012. Can seqlocks get along with programming language memory models?, in: Proceedings of the 2012 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness (MSPC), pp. 12–20.

Bootlin, 2026. Toolchains. `https://toolchains.bootlin.com`. Release 2026.08-1.

Buchsbaum, G., 1980. A spatial processor model for object colour perception. Journal of the Franklin Institute 310, 1–26.

Canny, J., 1986. A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-8, 679–698.

Dagum, L., Menon, R., 1998. OpenMP: An industry standard API for shared-memory programming. IEEE Computational Science and Engineering 5, 46–55.

Dronecode Foundation, 2026. QGroundControl. https://qgroundcontrol.com.

Duda, R.O., Hart, P.E., 1973. Pattern Classification and Scene Analysis. Wiley, New York.

FFmpeg developers, 2026. FFmpeg. https://ffmpeg.org. Version 9.0.2.

Franke, H., Russell, R., Kirkwood, M., 2002. Fuss, futexes and furwocks: Fast userlevel locking in Linux, in: Proceedings of the Ottawa Linux Symposium, pp. 479–495.

Hitam, M.S., Awalludin, E.A., Wan Yussof, W.N.J.H., Bachok, Z., 2013. Mixture contrast limited adaptive histogram equalization for underwater image enhancement, in: 2013 International Conference on Computer Applications Technology (ICCAT), pp. 1–5.

ITU-R, 2011. Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios. International Telecommunication Union.

ITU-T, 2003. Recommendation H.264: Advanced video coding for generic audiovisual services. International Telecommunication Union.

Jaffe, J.S., 1990. Computer modeling and the design of optimal underwater imaging systems. IEEE Journal of Oceanic Engineering 15, 101–111.

Marr, D., Hildreth, E., 1980. Theory of edge detection. Proceedings of the Royal Society of London. Series B, Biological Sciences 207, 187–217.

Pizer, S.M., Amburn, E.P., Austin, J.D., Cromartie, R., Geselowitz, A., Greer, T., ter Haar Romeny, B., Zimmerman, J.B., Zuiderveld, K., 1987. Adaptive histogram equalization and its variations. Computer Vision, Graphics, and Image Processing 39, 355–368.

Raspberry Pi Ltd, 2026. rpicam-apps. https://github.com/raspberrypi/rpicam-apps.

Schettini, R., Corchs, S., 2010. Underwater image processing: State of the art of restoration and image enhancement methods. EURASIP Journal on Advances in Signal Processing 2010, 746052.

Schulzrinne, H., Casner, S., Frederick, R., Jacobson, V., 2003. RTP: A Transport Protocol for Real-Time Applications. Request for Comments 3550. Internet Engineering Task Force.

The libjpeg-turbo Project, 2026. libjpeg-turbo. https://libjpeg-turbo.org. Version 3.2.0.

The Linux kernel developers, 2026. Video for Linux API (V4L2). https://docs.kernel.org/userspace-api/media/v4l/v4l2.html.

VideoLAN, 2026. x264. https://www.videolan.org/developers/x264.html.

Wang, Y.K., Even, R., Kristensen, T., Jesup, R., 2011. RTP Payload Format for H.264 Video. Request for Comments 6184. Internet Engineering Task Force.

zmölnig, I.m., 2026. v4l2loopback. https://github.com/umlaeute/v4l2loopback.

Zuiderveld, K., 1994. Contrast limited adaptive histogram equalization, in: Heckbert, P.S. (Ed.), Graphics Gems IV. Academic Press, San Diego, pp. 474–485.